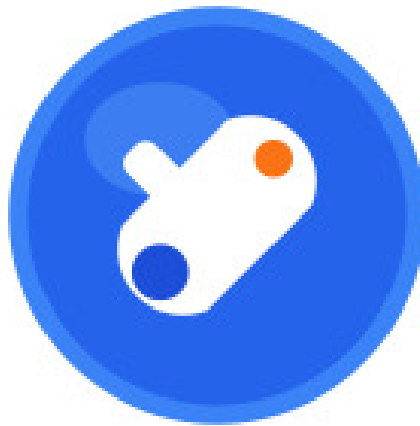




MISKOLCI SZC KANDÓ KÁLMÁN INFORMATIKAI TECHNIKUM



Az OnlyFix alkalmazás dokumentációja

Készítette

Dluhi Dániel

Luczi Alex

Szoftverfejlesztő- és tesztelő szak

Témavezető

Horváth István

konzulens

MISKOLC, 2026

Tartalomjegyzék

1. Bevezetés, a szoftver célja	3
2. Probléma — Megoldás	4
2.1. A probléma megfogalmazása	4
2.2. A megoldás	4
3. A program technológiai háttere	6
3.1. Vue.js és Inertia.js (webes kliens)	6
3.2. MAUI kliens alkalmazás	7
3.3. Laravel	8
3.4. MySQL adatbázis	9
3.5. phpMyAdmin	10
3.6. Mailpit	11
3.7. Docker	11
4. Fejlesztési eszközök és módszertan	12
4.1. Git	12
4.2. Trello	12
5. OnlyFix alkalmazás	13
5.1. Milyen részekből áll?	13
5.2. Felhasználók	13
5.3. Jegykezelés — a folyamat bemutatása	14
6. MAUI Admin kliens	15
6.1. Miért jó?	15
6.2. Kiknek szól?	15
6.3. A folyamat bemutatása	15
Irodalomjegyzék	18

1. fejezet

Bevezetés, a szoftver célja

Az OnlyFix szoftvercsomag egy nyílt forráskódú, MIT licenc alatt publikált járműszervizmenedzsment rendszer [2], amelynek célja a hagyományos, papíralapú szervizkezelési folyamatok kiváltása egy egységes, digitális jegykezelő (*ticketing*) platformmal [1]. A rendszer lehetővé teszi, hogy a gépjármű-tulajdonosok, szerelők és rendszerüzemeltetők hatékonyan kezeljék a járművekkel kapcsolatos hibajegyeket webes valamint natív asztali és mobil kliensről egyaránt. A szoftvercsomag két fő komponensből áll:

- **OnlyFix webalkalmazás** (randomUSR56/onlyfix) — egy Laravel 12 keretrendszerre épülő monolitikus alkalmazás Vue.js 3 frontend réteggel és Inertia.js összekötő réteggel, valamint egy RESTful API réteggel külső kliensek kiszolgálására.
- **Admin MAUI kliens alkalmazás** (randomUSR56/Admin) — egy .NET 10 MAUI keretrendszerre épülő natív kliens alkalmazás, amely a Laravel backend RESTful API végpontjaira támaszkodva biztosít adminisztrációs hozzáférést.

2. fejezet

Probléma — Megoldás

2.1. A probléma megfogalmazása

A gépjármű-szervizek mindennapi működése során az ügyfelek jellemzően telefonon vagy személyesen jelentik be a járműveikkel kapcsolatos problémákat. Ez a folyamat számos hatékonysági problémát vet fel: a bejelentések nehezen nyomon követhetők, az állapotváltozásokról nincs automatikus értesítés, a szerelők munkaelosztása manuálisan történik, és a történeti adatok visszakeresése körülményes. Továbbá a szerviz üzemeltetőinek nincs egységes felülete a rendszer átfogó monitorozásához.

2.2. A megoldás

Az OnlyFix szoftvercsomag egy háromszintű szerepkör-alapú hozzáférés-vezérlési rendszert (angolul: *Role-Based Access Control*, röviden RBAC) [3] valósít meg, amelyben:

- **Felhasználó (User)**: regisztrálhatja járműveit, hibajegyeket nyithat, nyomon követheti azok állapotát, és lezárhatja a teljesített jegyeket.
- **Szerelő (Mechanic)**: megtekintheti az összes nyitott jegyet, elfogadhatja a munkát, frissítheti a jegy állapotát, és hozzáférhet a statisztikai kimutatásokhoz.
- **Adminisztrátor (Admin)**: teljes körű rendszer-hozzáféréssel rendelkezik, beleértve a felhasználókezelést, a problémakatalógus karbantartását és a rendszer konfigurációját.

A hibajegy életciklusa egy jól definiált munkafolyamatot (angolul: *workflow*) követ:

Nyitott (Open) → Kiosztott (Assigned) → Folyamatban (In Progress)
→ Befejezett (Completed) → Lezárt (Closed)

Minden állapotátmenet időbélyeggel (angolul: *timestamp*) van ellátva, és egy jegyhez több probléma is társítható az előre definiált problémakatalógusból.

3. fejezet

A program technológiai háttere

3.1. Vue.js és Inertia.js (webes kliens)

A Vue.js egy progresszív JavaScript keretrendszer (angolul: *progressive framework*), amelyet Evan You fejlesztett ki 2014-ben [4]. A „progresszív” jelző arra utal, hogy a keretrendszer fokozatosan adoptálható: egyszerű projektekben minimális konfigurációval használható, míg komplex egyoldalas alkalmazások (angolul: *Single-Page Application*, röviden SPA) fejlesztéséhez is teljes körű eszköztárat biztosít. A projektben a Vue.js 3.5.13-as verzióját alkalmazzuk, amely a Composition API-t helyezi előtérbe a korábbi Options API-val szemben, lehetővé téve a reaktív logika moduláris és újrafelhasználható szervezését.

Az Inertia.js egy úgynevezett „modern monolit” architektúrát megvalósító könyvtár, amely Jonathan Reinink nevéhez fűződik [5]. Az Inertia.js nem egy klasszikus JavaScript keretrendszer, hanem egy összekötő réteg (angolul: *glue layer*), amely lehetővé teszi, hogy egy szerver oldali keretrendszer — jelen esetben a Laravel — közvetlenül Vue.js komponenseket renderelhessen anélkül, hogy külön RESTful API-t kellene építeni a webes frontend kiszolgálásához. Az Inertia.js ezt úgy valósítja meg, hogy az első oldalbetöltéskor egy hagyományos, szerver által renderelt HTML-választ küld (angolul: *Server-Side Rendering*, röviden SSR), majd az ezt követő navigációknál XHR (XMLHttpRequest) kérésekkel JSON formátumú adatokat cserél a szerver és a kliens között, így SPA-szerű felhasználói élményt biztosítva [5].

Ez a megoldás lényeges különbséget jelent a MAUI Admin klienshez képest: míg a Vue.js + Inertia.js webes kliens közvetlenül a Laravel web.php útvonalait használja és az Inertia protokollon keresztül kommunikál a szerverrel, addig a MAUI kliens kizárólag a routes/api.php fájlban definiált RESTful API végpontokra küld HTTP kéréseket, és a válaszokat JSON formátumban dolgozza fel.

A webes frontend működési mechanizmusa a következő: a Laravel szerver a routes/web.php fájlban definiált útvonalak mentén Inertia-válaszokat generál az Iner-

`render()` metódus segítségével. Ez a metódus a megadott Vue.js komponens nevét és a hozzá tartozó adatokat (angolul: *props*) egy JSON struktúrában csomagolja, amelyet az Inertia.js kliens oldali adaptore (@inertiajs/vue3) fogad és a megfelelő Vue.js komponensbe injektál. A fejlesztői környezetben a Vite 7.0.4 build eszköz biztosítja a HMR (Hot Module Replacement) funkciót, amely lehetővé teszi a kód módosításainak azonnali, böngésző-frissítés nélküli megjelenítését [6].

A frontend felhasználói felülete a Tailwind CSS 4.1.1 utility-first CSS keretrendszerre épül [7], kiegészítve a Reka UI 2.4.1 headless komponenskönyvtárral, amely akadálymentes (angolul: *accessible*), stílusmentes UI primitíveket biztosít. A TypeScript 5.2.2 típusrendszer használata a fejlesztés során biztosítja a típusbiztonságot (angolul: *type safety*), csökkentve a futásidejű hibák előfordulásának valószínűségét.

3.2. MAUI kliens alkalmazás

Az Admin MAUI kliens alkalmazás a szoftvercsomag második frontendkomponense, amely kizárólag rendszerüzemeltetők és adminisztrátorok számára készült. Feladata, hogy natív asztali (Windows, macOS) és mobil (Android, iOS) platformokon biztosítson hozzáférést az OnlyFix rendszer adminisztrációs funkcióihoz. Az alkalmazás MVVM (Model-View-ViewModel) architektúrais mintát követ a CommunityToolkit.Mvvm 8.4.0 csomag segítségével [8], és a .NET beépített függőséginjektálási (angolul: *Dependency Injection*, röviden DI) rendszerét használja a szolgáltatások és nézetmodellek (angolul: *ViewModel*) kezelésére.

A MAUI kliens a Laravel backend routes/api.php fájljában definiált RESTful API végpontokkal kommunikál HTTP protokollon keresztül. Az alkalmazás egy típusos HttpClient-et használ (angolul: *typed HttpClient*), amelyet a MauiProgram.cs fájlban regisztrál a DI konténerbe a `builder.Services.AddHttpClient<IApiClient, ApiClient>()` metódussal. Az alap URL (`http://onlyfix.local`) a Laravel backend elérési pontjára mutat.

Az autentikáció token-alapú: a felhasználó a POST /api/login végpontra hitelesíti magát, és a visszakapott Bearer token-t az AuthTokenStore singleton szolgáltatás tárolja. Ezt követően minden API-hívás az Authorization: Bearer {token} HTTP fejléccel kerül elküldésre. Ha a szerver 401 Unauthorized státuszkóddal válaszol, az alkalmazás automatikusan törli a tárolt token-t és a bejelentkezési oldalra navigál.

A navigációs struktúrát a Shell komponens biztosítja, amely egy Flyout oldalsávval rendelkezik a következő szekciókkal: Dashboard (Írányítópult), Users (Felhasználók), Cars (Járművek), Problems (Hibák) és Tickets (Szervizjegyek). A részletes nézetek (pl. felhasználó szerkesztése, jegy részletei) push-navigációval érhetők el a Shell útvonal-rendszerén keresztül.

A .NET MAUI (Multi-platform App UI) a Microsoft által fejlesztett, nyílt forrás-

kódú, platformfüggetlen alkalmazásfejlesztési keretrendszer, amely a Xamarin.Forms utódja [9]. A MAUI legfőbb előnye, hogy egyetlen C# kódbázisból natív alkalmazásokat készíthetünk Windows, macOS, Android és iOS platformokra egyaránt. A projekt a .NET 10-es verzióra épül, amely a legújabb LTS (Long-Term Support) kiadás [10].

A MAUI választását az indokolta, hogy az adminisztrációs kliens számára előnyös a natív platformintegráció (pl. operációs rendszer szintű értesítések, fájlrendszer-hozzáférés), miközben a közös kódbázis jelentősen csökkenti a fejlesztési és karbantartási költségeket.

Fontos kiemelni a két frontend kliens kommunikációs modelljének alapvető különbségét. A Vue.js + Inertia.js webes kliens az úgynevezett „modern monolit” architektúrát követi: a böngésző a Laravel szerver routes/web.php fájljában definiált útvonalakra küld kéréseket, és az Inertia.js protokoll révén a szerver közvetlenül Vue.js komponenseket és a hozzájuk tartozó adatokat szolgáltatja ki — ez a megoldás nem igényel külön API réteget a webes frontend számára [5].

Ezzel szemben a MAUI kliens egy teljesen önálló, natív alkalmazás, amely kizárólag a Laravel backend routes/api.php fájljában definiált REST (Representational State Transfer) [11] végpontokra támaszkodik. A kommunikáció tisztán HTTP-alapú, JSON formátumú kérés-válasz párokat használ, és a hitelesítés Laravel Sanctum token-alapú mechanizmussal történik [12]. Ez a szétválasztás (angolul: *separation of concerns*) lehetővé teszi, hogy a MAUI kliens teljesen független legyen a webes frontendtől, és a backendhez való csatlakozáshoz mindössze az API alap URL-jének és a hitelesítő tokennek az ismeretére van szükség.

3.3. Laravel

A Laravel egy PHP nyelven írt, nyílt forráskódú, szervermotoron (angolul: *server-side*) futó webalkalmazás-keretrendszer, amelyet Taylor Otwell fejlesztett ki, és amely az MVC (Model-View-Controller) architekturális mintát követi [13]. A Laravel 12-es verzióját alkalmazzuk a projektben, amely a PHP 8.2 vagy újabb verziójával kompatibilis.

A Laravel választását több tényező indokolta:

- **Eloquent ORM** (Object-Relational Mapping): az aktív rekord (angolul: *Active Record*) mintát megvalósító adatbázis-absztrakciós réteg, amely PHP objektumokként kezeli az adatbázis tábláit és rekordjait, így a fejlesztőnek nem szükséges nyers SQL lekérdezéseket írnia a legtöbb műveletnél [14].
- **Artisan CLI** (Command-Line Interface): parancssori eszköz, amely automatizálja a fejlesztési feladatokat (migrációk futtatása, tesztadatok generálása, útvonalak listázása stb.) [13].

- **Beépített autentikációs rendszer:** a Laravel Fortify és Sanctum csomagok révén egyszerre támogat SPA-alapú munkamenet-hitelesítést (angolul: *session-based authentication*) és API token-alapú hitelesítést.
- **Migráció-alapú sémadefiníció:** az adatbázis sémájának verziózott, programatikus leírása, amely biztosítja a fejlesztési környezetek közötti konzisztenciát [13].
- **Kiterjedt ökoszisztéma:** a Composer csomagkezelő révén elérhető harmadik féltől származó csomagok (pl. Spatie Laravel Permission a szerepkör-kezeléshez [15]).

A Laravel alkalmazás a kérések fogadásától a válasz kiküldéséig egy jól definiált életciklust (angolul: *request lifecycle*) követ [13]:

1. A webservert (jelen esetben Nginx) fogadja a HTTP kérést és továbbítja a PHP-FPM (FastCGI Process Manager) folyamatnak.
2. A Laravel bootstrapper betölti a konfigurációs fájlokat és az alkalmazás szolgáltatásait (angolul: *Service Providers*).
3. A kérés áthalad a köztes szoftverek (angolul: *middleware*) láncolatán (pl. CSRF-védelem, autentikáció, jogosultság-ellenőrzés).
4. Az útválasztó (angolul: *Router*) a kérést a megfelelő vezérlőnek (angolul: *Controller*) továbbítja.
5. A vezérlő az Eloquent ORM-en keresztül lekérdezi vagy módosítja az adatbázist, majd a választ visszaadja.
6. Webes útvonalak esetén a válasz egy Inertia-válasz (Vue.js komponens + adatok); API útvonalak esetén JSON formátumú válasz.

A projektben az alkalmazáslogika az `app/` könyvtárban található, benne az `Http/Controllers/Api/` alkönyvtárban az API vezérlők (`AuthController`, `CarController`, `ProblemController`, `TicketController`, `UserController`), a `Models/` könyvtárban az Eloquent modellek (`User`, `Car`, `Problem`, `Ticket`), a `Policies/` könyvtárban az erőforrás-szintű jogosultsági politikák (angolul: *Policy*).

3.4. MySQL adatbázis

Az OnlyFix alkalmazás MySQL 8.0 relációs adatbázis-kezelő rendszert használ az InnoDB tárolómotorral (angolul: *storage engine*) [16]. Az InnoDB a MySQL alapértelmezett tárolómotorja, amely ACID (Atomicity, Consistency, Isolation, Durability) tulajdonságokat biztosít a tranzakciók számára [16]. Az ACID tulajdonságok garantálják, hogy az adatbázis-műveletek atomiak (vagy teljes egészében végrehajthatók, vagy

egyáltalán nem), konzisztensek (a tranzakció előtti és utáni állapot egyaránt érvényes), izoláltak (a párhuzamosan futó tranzakciók nem zavarják egymást) és tartósak (a véglegesített tranzakciók eredménye nem veszik el rendszerhiba esetén sem).

A MySQL választását az indokolta, hogy a Laravel keretrendszer natív támogatást biztosít hozzá, kiterjedt közösségi és vállalati támogatottságú, és a Docker-alapú fejlesztői környezetben egyszerűen konfigurálható. A MySQL 8.0-ás verziója továbbá támogatja a JSON típusú oszlopokat, az ablakfüggvényeket (angolul: *window functions*) és a közös tábla kifejezéseket (angolul: *Common Table Expressions*, röviden CTE) [16].

Az adatbázis séma az alábbi fő entitásokat és relációkat tartalmazza:

- **users**: felhasználók adatai (id, név, e-mail, jelszó, kétfaktoros hitelesítési mezők, időbélyegek). Relációk: `hasMany` → `cars`, `tickets`; `hasMany` → `assignedTickets` (mint szerelő).
- **cars**: járművek adatai (id, `user_id`, gyártmány, modell, évjárat, rendszám, alvázszám, szín). Relációk: `belongsTo` → `user`; `hasMany` → `tickets`.
- **problems**: problémakatalógus (id, név, kategória [`engine`, `transmission`, `electrical`, `brakes`, `suspension`, `steering`, `body`, `other`], leírás, aktív státusz). Relációk: `belongsToMany` → `tickets` (pivot táblán keresztül).
- **tickets**: szervizjegyek (id, `user_id`, `mechanic_id`, `car_id`, állapot, prioritás [`low`, `medium`, `high`, `urgent`], leírás, elfogadás időpontja, befejezés időpontja). Relációk: `belongsTo` → `user`, `mechanic`, `car`; `belongsToMany` → `problems`.
- **ticket_problems**: pivot tábla (id, `ticket_id`, `problem_id`, megjegyzések). Egyedi megszorítás (angolul: *unique constraint*) a [`ticket_id`, `problem_id`] pároson.
- **roles**, **permissions** és kapcsolódó pivot táblák: a Spatie Laravel Permission csomag által kezelt RBAC struktúra [15].

3.5. phpMyAdmin

A phpMyAdmin egy nyílt forráskódú, PHP-ben írt webes adatbázis-adminisztrációs eszköz, amely grafikus felhasználói felületet biztosít a MySQL adatbázis kezeléséhez [17]. A fejlesztési folyamat során a phpMyAdmin lehetővé teszi az adatbázis tartalmának vizuális böngészését, SQL lekérdezések futtatását, és a táblák struktúrájának ellenőrzését anélkül, hogy parancssori eszközöket kellene használni.

A phpMyAdmin a Docker Compose konfigurációban egy önálló konténerként fut (`onlyfix_phpmyadmin`), és a `http://phpmyadmin.onlyfix.local:8080` címen érhető el. A konténer automatikusan csatlakozik a MySQL konténerhez a belső Docker hálózaton keresztül.

3.6. Mailpit

A Mailpit egy könnyűsúlyú, nyílt forráskódú e-mail tesztelő eszköz, amely egy helyi SMTP szervert és egy webes felületet biztosít a fejlesztés során küldött e-mailek elfogadásához és megtekintéséhez [18]. A Mailpit használatával a fejlesztők anélkül tesztelhetik az e-mail küldési funkcionalitást, hogy valódi e-mail szolgáltatóhoz kellene csatlakozniuk.

A Mailpit a Docker Compose konfigurációban egy önálló konténerként fut, amely az SMTP forgalmat az 1025-ös porton fogadja, a webes felület pedig a 8025-ös porton érhető el (<http://mailpit.onlyfix.local:8025>). A Laravel .env konfigurációjában az SMTP szerver címeként a Mailpit konténer belső IP-címe van megadva, így minden, az alkalmazás által küldött e-mail a Mailpit felületén jelenik meg.

3.7. Docker

A Docker egy nyílt forráskódú konténerizációs platform, amely lehetővé teszi alkalmazások és azok függőségeinek egységes csomagolását, szállítását és futtatását úgynevezett konténerekben (angolul: *containers*) [19]. A konténerek a virtuális gépekhez hasonlóan izolált futtatási környezetet biztosítanak, de lényegesen kisebb erőforrás-igénnyel, mivel az operációs rendszer kernelét megosztva használják.

A Docker használata a fejlesztési környezet reprodukálhatóságát biztosítja: minden fejlesztő azonos operációs rendszertől független környezetben dolgozhat, függetlenül a saját operációs rendszerétől. A „works on my machine” probléma kiküszöbölése mellett a Docker lehetővé teszi a szolgáltatások (webszerver, adatbázis, e-mail szerver, cache) egymástól független skálázását és kezelését.

Az OnlyFix projekt egy docker-compose.yml fájlt használ, amely az alábbi hét szolgáltatás-konténert definiálja:

Konténer neve	Szolgáltatás	Leírás
onlyfix_app	PHP-FPM	Laravel backend alkalmazás
onlyfix_nginx	Nginx	Webszerver, reverse proxy
onlyfix_node	Node.js 20	Vite fejlesztői szerver (HMR)
onlyfix_db	MySQL 8.0	Relációs adatbázis
onlyfix_mailpit	Mailpit	E-mail tesztelés
onlyfix_queue	PHP-FPM	Laravel Queue Worker
onlyfix_phpmyadmin	phpMyAdmin	Adatbázis-adminisztráció

3.1. táblázat. Docker szolgáltatás-konténerek

A konténerek egy belső Docker bridge hálózaton (172.20.0.0/16 alhálózat) kommunikálnak egymással, rögzített IP-címekkel.

4. fejezet

Fejlesztési eszközök és módszertan

4.1. Git

A Git egy elosztott verziókezelő rendszer (angolul: *Distributed Version Control System*, röviden DVCS), amelyet Linus Torvalds fejlesztett ki 2005-ben [20]. A Git lehetővé teszi a forráskód változásainak nyomon követését, a fejlesztési ágak (angolul: *branches*) párhuzamos kezelését, valamint a csapattagok közötti együttműködést konfliktuskezelési (angolul: *merge conflict resolution*) mechanizmusokkal.

A verziókezelés Git segítségével történik, a forráskód a GitHub platformon van tárolva két különálló repositoryban. A fejlesztési munkafolyamat a *feature branch* stratégiát követi: minden új funkció vagy hibajavítás egy külön ágon készül el, majd pull request formájában kerül beolvasztásra az integráló ágba. Az OnlyFix repository dev ága szolgál a legfrissebb fejlesztési változtatások integrációs ágaként, míg a main ág a stabil kiadásokat tartalmazza. A dev ág védett (angolul: *protected branch*), ami azt jelenti, hogy közvetlen push műveletekkel nem módosítható — a változtatások kizárólag pull request-eken (angolul: *lehúzási kérelmek*) keresztül kerülhetnek be, amelyeket legalább egy másik fejlesztőnek jóvá kell hagynia. Az Admin repository a master ágot használja alapértelmezett ágként.

4.2. Trello

A feladatkezelés és a projektmenedzsment a Trello platformon keresztül történik, amely egy Kanban-alapú projektvezető eszköz. A Trello táblákon a feladatok állapota (pl. „Teendő”, „Folyamatban”, „Átvizsgálásra vár”, „Kész”) vizuálisan nyomon követhető, és a fejlesztési sprintek tervezése is itt zajlik.

5. fejezet

OnlyFix alkalmazás

5.1. Milyen részekből áll?

Az OnlyFix alkalmazás az alábbi fő modulokból épül fel:

- **Autentikációs modul:** felhasználók regisztrációja, bejelentkezés, kijelentkezés, jelszókezelés. A Laravel Fortify és Sanctum csomagokra épül, és támogatja a kétfaktoros hitelesítést (angolul: *Two-Factor Authentication*, röviden 2FA).
- **Járműkezelő modul:** a felhasználók regisztrálhatják járműveiket (gyártmány, modell, évjárat, rendszám, alvázszám, szín), és megtekinthetik az adott járműhöz tartozó szervizjegyeket.
- **Problémakatalógus modul:** 56 előre definiált járműhiba 8 kategóriában (motor, sebességváltó, elektromos, fékek, futómű, kormányzás, karosszéria, egyéb). A szerelők és adminisztrátorok bővíthetik, módosíthatják a katalógust.
- **Jegykezelő (ticketing) modul:** a rendszer központi eleme, ahol a felhasználók szervizjegyeket hozhatnak létre, a szerelők elfogadhatják és feldolgozhatják azokat, és a felhasználók lezárhatják a teljesített jegyeket.
- **Statisztikai modul:** valós idejű statisztikák a jegyek állapotáról, a leggyakoribb problémákról, a szerelők munkaelosztásáról.
- **Felhasználókezelő modul (adminisztrátor):** felhasználók listázása, létrehozása, módosítása, törlése, szerepkörök hozzárendelése.

5.2. Felhasználók

A rendszer három felhasználói szerepkört definiál, amelyeket a Spatie Laravel Permission csomag kezel [\[15\]](#):

Szerepkör	Jogosultságok
Felhasználó	Saját járművek és jegyek kezelése, szerviz előzmények megtekintése
Szerelő	Összes jegy megtekintése, munka elfogadása, állapotok frissítése
Adminisztrátor	Teljes rendszer-hozzáférés, felhasználókezelés, rendszerkonfiguráció

5.1. táblázat. Felhasználói szerepkörök és jogosultságok

5.3. Jegykezelés — a folyamat bemutatása

A szervizjegy életciklusa a következő lépéseket foglalja magában:

1. **Jegy létrehozása (Open):** A felhasználó kiválasztja az érintett járművet, megadja a prioritást (alacsony, közepes, magas, sürgős), szöveges leírást ír, és a problémakatalógusból kiválasztja a releváns problémákat. A jegy „Nyitott” állapotban jön létre.
2. **Jegy elfogadása (Assigned):** Egy szerelő megtekinti a nyitott jegyeket és elfogadja a munkát a POST /tickets/{id}/accept végponton. A jegy állapota „Kiosztott”-ra változik, és a mechanic_id mező kitöltődik.
3. **Munka megkezdése (In Progress):** A szerelő jelzi, hogy megkezdte a munkát a POST /tickets/{id}/start végponton. Az állapot „Folyamatban”-ra változik.
4. **Munka befejezése (Completed):** A szerelő jelzi a munka befejezését a POST /tickets/{id}/complete végponton. Az állapot „Befejezett”-re változik, és a completed_at időbélyeg kitöltődik.
5. **Jegy lezárása (Closed):** A felhasználó (vagy adminisztrátor) lezárja a jegyet a POST /tickets/{id}/close végponton, ezzel nyugtázva a szerviz teljesítését.

6. fejezet

MAUI Admin kliens

6.1. Miért jó?

Az Admin MAUI kliens alkalmazás egy natív, platformfüggetlen asztali és mobil alkalmazás, amely az OnlyFix rendszer teljes adminisztrációs funkcionalitását egységes felületen teszi elérhetővé. A natív alkalmazás előnye a webes felülettel szemben, hogy gyorsabb válaszidőket, platformspecifikus felhasználói élményt és offline-képességeket biztosíthat. Az MVVM architektúra révén a kód jól tesztelhető és karbantartható.

6.2. Kiknek szól?

Az Admin kliens elsősorban rendszerüzemeltetőknek, szervizvezetőknek és adminisztrátoroknak készült, akiknek átfogó rálátásra van szükségük a rendszer teljes működésére. A kliens lehetővé teszi:

- A felhasználók, járművek, problémák és jegyek teljes körű kezelését (CRUD műveletek).
- Az irányítópulton (angolul: *Dashboard*) valós idejű statisztikák megtekintését.
- A jegy-munkafolyamat állapotátmeneteinek kezelését.
- A rendszer egészségének monitorozását az API health endpoint segítségével.

6.3. A folyamat bemutatása

1. **Bejelentkezés:** az alkalmazás indításakor az AuthTokenStore ellenőrzi, hogy létezik-e tárolt Bearer token. Ha igen, a felhasználó közvetlenül az irányítópultra kerül; ha nem, a bejelentkezési oldalra navigál. A bejelentkezés a POST `/api/login` végpontra küldött kéréssel történik.

2. **Navigáció:** a bejelentkezést követően a felhasználó a bal oldali Flyout navigációs sáv segítségével válthat a Dashboard, Users, Cars, Problems és Tickets szekciók között.
3. **CRUD műveletek:** minden szekció listanézetben jeleníti meg az entitásokat, amelyekre kattintva részletes nézet nyílik (push navigáció). Az entitások szerkeszthetők és törölhetők; új entitások az adott szekció létrehozási formján keresztül hozhatók létre.
4. **Automatikus kijelentkezés:** ha bármely API hívás 401 Unauthorized választ kap, az alkalmazás automatikusan törli a tárolt tokent és a bejelentkezési oldalra navigál.

Köszönetnyilvánítás

Irodalomjegyzék

- [1] R. T. Fielding, „Architectural Styles and the Design of Network-based Software Architectures,” University of California, Irvine, 2000.
- [2] Open Source Initiative, „The MIT License,” [Online]. Available: <https://opensource.org/licenses/MIT>. [Hozzáférés dátuma: 26 02 2026].
- [3] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn és R. Chandramouli, „Proposed NIST Standard for Role-Based Access Control,” *ACM Transactions on Information and System Security*, vol. 4, no. 3, pp. 224–274, 2001.
- [4] Vue.js, „Vue.js — The Progressive JavaScript Framework,” [Online]. Available: <https://vuejs.org/guide/introduction.html>. [Hozzáférés dátuma: 26 02 2026].
- [5] Inertia.js, „Inertia.js — The Modern Monolith,” [Online]. Available: <https://inertiajs.com/>. [Hozzáférés dátuma: 26 02 2026].
- [6] Vite, „Vite — Next Generation Frontend Tooling,” [Online]. Available: <https://vitejs.dev/guide/>. [Hozzáférés dátuma: 26 02 2026].
- [7] Tailwind Labs, „Tailwind CSS — Rapidly build modern websites without ever leaving your HTML,” [Online]. Available: <https://tailwindcss.com/docs>. [Hozzáférés dátuma: 26 02 2026].
- [8] Microsoft, „MVVM Toolkit — .NET Community Toolkit,” [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/>. [Hozzáférés dátuma: 26 02 2026].
- [9] Microsoft, „What is .NET MAUI?,” [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/maui/what-is-maui>. [Hozzáférés dátuma: 26 02 2026].
- [10] Microsoft, „.NET 10 — What’s New,” [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview>. [Hozzáférés dátuma: 26 02 2026].

- [11] R. T. Fielding, „Representational State Transfer (REST),” in *Architectural Styles and the Design of Network-based Software Architectures*, Irvine, University of California, Irvine, 2000, Chapter 5.
- [12] Laravel, „Laravel Sanctum — API Token Authentication,” [Online]. Available: <https://laravel.com/docs/12.x/sanctum>. [Hozzáférés dátuma: 26 02 2026].
- [13] Laravel, „Laravel — The PHP Framework for Web Artisans,” [Online]. Available: <https://laravel.com/docs/12.x>. [Hozzáférés dátuma: 26 02 2026].
- [14] Laravel, „Eloquent: Getting Started,” [Online]. Available: <https://laravel.com/docs/12.x/eloquent>. [Hozzáférés dátuma: 26 02 2026].
- [15] Spatie, „Laravel Permission — Associate users with roles and permissions,” [Online]. Available: <https://spatie.be/docs/laravel-permission/v6/introduction>. [Hozzáférés dátuma: 26 02 2026].
- [16] Oracle Corporation, „MySQL 8.0 Reference Manual — InnoDB Storage Engine,” [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/innodb-storage-engine.html>. [Hozzáférés dátuma: 26 02 2026].
- [17] phpMyAdmin, „phpMyAdmin — Bringing MySQL to the web,” [Online]. Available: <https://www.phpmyadmin.net/docs/>. [Hozzáférés dátuma: 26 02 2026].
- [18] Mailpit, „Mailpit — Email and SMTP testing tool with API for developers,” [Online]. Available: <https://github.com/axllent/mailpit>. [Hozzáférés dátuma: 26 02 2026].
- [19] Docker, Inc., „Docker Documentation — What is a container?,” [Online]. Available: <https://docs.docker.com/get-started/overview/>. [Hozzáférés dátuma: 26 02 2026].
- [20] Git, „Git — Distributed Version Control System,” [Online]. Available: <https://git-scm.com/doc>. [Hozzáférés dátuma: 26 02 2026].